# Score Arrays and Snapshots

**Due date:** Posted in GeorgiaVIEW.

Download Assignment02_Package.zip and work in Student_Files. Complete the three tasks in order. Write one part, compile it, and run its checks before adding the next part.

The program reads scores, changes one entry, and keeps a copy of the original scores. In Task A, write the functions. In Task B, use them in the working-array program. In Task C, add the copy and a five-point curve.

## Task A Write Score Functions

Write the two function bodies in score_functions.cpp. Keep the headers below. The supplied check program provides main, so do not add main to this file.

```
int maxScore(const int* scores, int count);
bool updateScore(int* scores, int count, int index, int replacement);
```

### maxScore

Return -1 if scores is nullptr or count <= 0. Otherwise return the largest int in scores[0] through scores[count - 1]. A valid array may contain negative integers. Do not change or release the array.

### updateScore

Return false if scores is nullptr, count <= 0, index is outside [0, count), or replacement is outside [0, 100]. Leave every element unchanged on failure. Otherwise change exactly scores[index] and return true. Do not release the array.

For valid calls with a non-null pointer and positive count, assume the pointer refers to at least count live int elements. Checking a pointer against nullptr does not check its lifetime or the array size.

### Check Task A

```
make 1
```

The supplied checks print PASS or FAIL for each group. Each group starts with fresh arrays. The two updates in A04 run consecutively on one array.

| Check | Inputs and expected result |
|---|---|
| A01 | 72 95 55 80 -> maximum 95 |
| A02 | Null/nonpositive count -> -1; one score 0 -> 0; -9 -3 -12 -> -3 |
| A03 | 10 20 30; index 1, value 88 -> 10 88 30; return true |
| A04 | 10 20 30; set index 0 to 0, then index 2 to 100 -> 0 20 100 |
| A05 | Reject a null array, count <= 0, index -1 or count, and values -1 or 101; no write |

# Task B Write the Working Array Program

Write main in score_working.cpp. It uses your Task A functions. Keep the supplied printScores function. Add no input prompts.

## Input and program order

Read count. Accept 1 through 20. Allocate exactly count value-initialized int elements after count is valid.

Read count scores. Accept 0 through 100 for each score. Then read a zero-based correction index and a replacement score.

Call updateScore and store its Boolean result. Print the report below. Release the working array with delete[], set working to nullptr, and print the final owner check.

A parsed but rejected correction still produces the report. Its Boolean result is false and the scores are unchanged.

## Example

Input:

```
4
72 95 55 80
2 88
```

Output:

```
correction applied: true
working scores: 72 95 88 80
working max: 95
owner cleared: true
```

## Input errors

| Input problem | Print exactly | Cleanup before returning |
|---|---|---|
| Unreadable count or count outside 1-20 | Invalid score count | No array has been allocated |
| Missing/unreadable score or score outside 0-100 | Invalid score data | Release working and set it to nullptr |
| Missing/unreadable correction index or replacement | Invalid correction data | Release working and set it to nullptr |

On an input error, print the one error message and return 0. Do not print the normal report. Whitespace separates input values. Assume all allocations succeed; allocation-failure handling is not part of this assignment.

## Check Task B

```
make 2
```

This runs every supplied input and compares the output with expected/working. Fix a failed case, then run make 2 again.

## Task C Add the Snapshot and Curve

Use your completed Task B main as the starting point in score_snapshot.cpp. Add the two functions below and update main. Keep printScores and the Task A function headers.

```
int* cloneScores(const int* source, int count);
void curveScores(int* scores, int count);
```

### cloneScores

Return nullptr for a null source or count <= 0. Otherwise allocate a separate int[count], copy every element, and return its pointer. Do not change or release source. The caller owns the returned array.

### curveScores

For nullptr or count <= 0, return without writing. Otherwise add 5 to each score, with a maximum of 100. This function borrows the array; it does not release it. Valid input scores are in [0, 100].

### Changes to main

Keep the Task B input checks. After both correction tokens have been read successfully, call cloneScores before updateScore. Store the copy in snapshot.

Apply the correction and print the same first three report lines as Task B. Then call curveScores on working and print curved scores and curved max. Apply the curve even if updateScore returned false.

Release working and set it to nullptr. Print snapshot and its maximum after that release. Finally release snapshot, set it to nullptr, and print the final owner check. Every error path must release any array already allocated.

### Output for the Task B example

```
correction applied: true
working scores: 72 95 88 80
working max: 95
curved scores: 77 100 93 85
curved max: 100
snapshot after working release: 72 95 55 80
snapshot max: 95
owners cleared: true
```

The snapshot is 72 95 55 80 throughout. The correction and curve change only working. The two pointers own separate arrays; clearing a pointer alone does not release its array.

### Check Task C

```
make 3
```

This compares the output with expected/snapshot. Use 08_curve_limit.txt to check 94 -> 99, 95 -> 100, and 100 -> 100. Use the same input-error messages as Task B.

## Run and Check

Open the Ubuntu / WSL terminal in Student_Files. Use the same numbered make workflow used in class:

```
make list
make 1
make 2
make 3
```

The starters compile but have unfinished function bodies and main programs. Their checks will fail until you write the code. Repair compiler warnings as you work. Keep the supplied inputs, expected output, and support files unchanged.

### Supplied program inputs

Tasks B and C use the same input files. The complete output is in expected/working and expected/snapshot. Each input starts a new program run.

| Input file | What it checks |
|---|---|
| 01_normal.txt | Count 4; accepted correction; independent snapshot |
| 02_one_score.txt | Count 1; change 100 to 0; curve changes 0 to 5 |
| 03_rejected_index.txt | Index equals count; reject correction, then curve working |
| 04_twenty_scores.txt | Count 20; change the last element to 100 |
| 05_invalid_count.txt | Count 0; reject before allocating |
| 06_invalid_score.txt | Score 101; release working and stop |
| 07_missing_correction.txt | Missing replacement; release working and stop |
| 08_curve_limit.txt | 94, 95, 100; add 5 without exceeding 100 |

### Before submitting

Check the first and last valid indexes, invalid replacement values, and incomplete input. Check that every array you allocate has one matching delete[] on each applicable return path.

Use C++17, free functions, raw pointers, raw arrays, and console input. Do not add vector, classes, file I/O, exceptions, smart pointers, templates, variable-length arrays, or global mutable state.

## Submit

Run make clean. Push the assignment folder to the course GitHub repository and submit its URL in GeorgiaVIEW. Include these three source files:

```
score_functions.cpp
score_working.cpp
score_snapshot.cpp
```

If you used AI, complete the separate AI_REFLECTION.md. If you did not use AI, omit it or leave the template unchanged. No screenshots, chat transcripts, or separate test log are required.

The supplied Makefile and support files may remain. Do not commit executables or build artifacts.